



Week 1

Machine Learning Introduction

AUTHOR

MAKSYM BONDAR

11.11.2025

Kaunas

Contents

1	Exploratory data analysis	1
1.1	Dataset Description	1
1.2	Initial Data Inspection	1
1.3	Tasks	3
1.4	1. Working Days vs. Non-working Days	3
1.5	2. Average Rentals on Working vs. Non-working Days	3
1.6	3. Proportion of Bad-weather Days	3
1.7	4–5. Mean and Standard Deviation of Rentals for High- and Low-demand Days	4
1.8	6. Are High-demand Days Specific to Particular Seasons?	4
1.9	7. Rental Statistics by Season and Working Day	4
1.10	8. Which Group Has More High-demand Days?	4
1.11	9. Maximum Number of Rentals and Its Characteristics	5
1.12	10. Average Rentals for High vs. Low Demand by Season	5
2	Visual Data Analysis	6
2.1	Preparation of Categorical Data	6
2.2	Category Frequencies Across the Whole Dataset	6
2.3	Category Frequencies by Demand Level	7
2.4	Uniqueness and Frequency of Each Feature	9
2.5	Summary of Feature Types	9
2.6	Part 1: Basic Observations	10
2.6.1	Working vs. non-working days	10
2.6.2	Share of high-demand days	10
2.6.3	Temperature difference between high and low demand	11
2.6.4	Risk mapping for summer working days	11
2.6.5	Utilization feature	12
2.6.6	Outlier detection and cleaning	13
2.7	Part 2: Visual Data Analysis	13
2.7.1	Pearson correlation matrix	13
2.7.2	Distribution of rentals on working vs. non-working days	14
2.7.3	Spearman rank correlation	17
2.7.4	Monthly pattern of high demand	17
3	Classification, Decision Trees and k Nearest Neighbors	18
3.1	Train–Test Split	18
3.2	Decision Tree Classifier	18
3.3	Hyperparameter-Tuned Decision Tree	19
3.4	Random Forest Classifier	19
3.5	<i>k</i> -Nearest Neighbours Classifier	20

Title of the document	Contents
3.6 Decision Tree Regressor for Predicting cnt	20
3.7 Model Performance Comparison	21
3.8 Summary	21
4 Linear Classification	23
4.1 Ordinary Least Squares Linear Regression	23
4.2 Lasso Regression	24
4.3 Random Forest Regression	24
4.4 Model Comparison	24
4.5 Feature Importance Analysis	25
5 Bagging and Random Forest	26
5.1 Bootstrapping the Mean Temperature on High-Demand Days	27
5.2 Logistic Regression	27
5.3 Ensembles, Bootstrapping, and Bagging	28
5.4 Random Forest Classifier	29
5.5 Bagging with Logistic Regression	30
5.6 Comparison of Ensemble Methods	31
6 Feature Engineering and Feature Selection	32
6.1 Feature Extraction	32
6.2 Feature Transformations	32
6.3 Feature Selection	33
7 Applied Feature Engineering and Feature Selection	33
7.1 Date and Time Feature Extraction	33
7.2 Cyclical Encoding of Periodic Features	34
7.3 Distribution Transformations of the Target	34
7.4 Variance-Based Feature Selection (Synthetic Example)	35
7.5 Univariate Feature Selection (SelectKBest)	35
7.6 Model-Based Feature Selection with Random Forest	36
7.7 Summary	36
References	37

1 Exploratory data analysis

1.1 Dataset Description

The dataset used in this assignment contains daily bike rental records from a public bike-sharing system. It provides information about temporal factors, weather conditions, and user activity, allowing an analysis of how environmental and calendar-related variables influence bike usage. Each row represents one day between 2011 and 2012.

Main Features

Feature	Description
instant	Unique record index.
dteday	Calendar date of the observation.
season	Season of the year: 1 – spring, 2 – summer, 3 – fall, 4 – winter.
yr	Year indicator (0: 2011, 1: 2012).
mnth	Month of the year (1–12).
holiday	Public holiday indicator (1: holiday, 0: otherwise).
weekday	Day of the week (0–6).
workingday	Working day indicator (1: not weekend and not holiday).
weathersit	Weather situation category: 1 – clear or partly cloudy; 2 – mist or clouds; 3 – light rain or light snow; 4 – heavy rain, snow, or storm.
temp	Normalized temperature.
atemp	Normalized perceived (“feels like”) temperature.
hum	Normalized humidity level.
windspeed	Normalized wind speed.
casual	Number of casual (non-registered) users.
registered	Number of registered users.
cnt	Total number of bike rentals (target variable).

This dataset is suitable for exploratory data analysis, feature engineering, and modeling tasks aimed at predicting daily bike demand.

1.2 Initial Data Inspection

To get an overview of the structure of the dataset, we first inspected the column names, checked for missing values, and examined basic metadata about the DataFrame.

Listing column names

Listing 1: Inspecting column names of the dataset

```
1 data.columns
```

Listing 1 prints the names of all columns in the dataset. The output shows 17 columns:

- temporal / calendar features: `dteday`, `season`, `yr`, `mnth`, `weekday`, `holiday`, `workingday`;
- weather-related features: `weathersit`, `temp`, `atemp`, `hum`, `windspeed`;
- user counts: `casual`, `registered`;
- total demand: `cnt`;
- index and derived label: `instant`, `demand`.

This confirms that the dataset combines time information, environmental conditions, and rental counts, which are all relevant for demand modelling.

Checking missing values

Listing 2: Checking for missing values

```
1 data.isna().sum()
```

Listing 2 computes, for each column, the number of missing (NA) values. The result is zero for all 17 columns (`instant`, `dteday`, `season`, ..., `demand`), which means that this dataset does not contain any missing values. Therefore, no imputation or row/column removal is required at this stage.

DataFrame structure and types

Listing 3: Summary information about the DataFrame

```
1 data.info()
```

Listing 3 prints a concise summary of the DataFrame:

- The dataset has **731** rows (daily records) and **17** columns.
- All columns have **731 non-null** entries, confirming again that there are no missing values.
- Data types:
 - 11 columns are `int64` (e.g. `season`, `yr`, `mnth`, `weekday`, `casual`, `registered`, `cnt`);

-
- 4 columns are float64 (temp, atemp, hum, windspeed);
 - 2 columns are object (dteday and demand).

From this, we conclude that dteday should later be converted from object to a proper date/time type, and demand can be treated as a categorical target derived from the numerical variable cnt. This structural overview guides the subsequent steps in preprocessing and feature engineering.

1.3 Tasks

In this section, we answer 10 analytical questions about the bike-sharing dataset. Each subsection includes the code executed and a short interpretation of the result.

1.4 1. Working Days vs. Non-working Days

```
1 data["workingday"].value_counts()
```

Output: 500 working days, 231 non-working days.

This shows that the dataset is slightly imbalanced toward working days.

1.5 2. Average Rentals on Working vs. Non-working Days

```
1 avg_working = data[data["workingday"] == 1]["cnt"].mean()  
2 avg_nonworking = data[data["workingday"] == 0]["cnt"].mean()  
3 avg_working, avg_nonworking
```

Working-day average: **4584.82** Non-working-day average: **4330.17**

Working days have slightly higher rental activity.

1.6 3. Proportion of Bad-weather Days

```
1 bad_weather_prop = data[data["weathersit"].isin([3, 4]).shape[0] / data.shape  
  [0]  
2 bad_weather_prop
```

Result: **0.0287** (approx. 2.9%).

Only a small fraction of days have truly bad weather.

1.7 4–5. Mean and Standard Deviation of Rentals for High- and Low-demand Days

```
1 threshold = data["cnt"].median()
2 high = data[data["cnt"] > threshold]["cnt"]
3 low = data[data["cnt"] <= threshold]["cnt"]
4
5 print("High demand mean:", round(high.mean(), 2))
6 print("High demand std:", round(high.std(), 2))
7
8 print("Low demand mean:", round(low.mean(), 2))
9 print("Low demand std:", round(low.std(), 2))
```

High-demand mean: **6087.91**, std: **1092.55**

Low-demand mean: **2925.11**, std: **1143.11**

The separation between high and low demand is very clear.

1.8 6. Are High-demand Days Specific to Particular Seasons?

```
1 high_seasons = data[data["cnt"] > threshold]["season"].unique()
2 high_seasons
```

All seasons appear among high-demand days.

Thus, high demand is not limited to a single season.

1.9 7. Rental Statistics by Season and Working Day

```
1 for (season, working), sub_df in data.groupby(["season", "workingday"]):
2     print(f"Season: {season}, workingday: {working}")
3     print(sub_df["cnt"].describe())
4     print()
```

This output provides detailed distributions (mean, std, quartiles) for each seasonâworkingday pair. Summer (season 3) generally shows the highest rental counts, especially on working days.

1.10 8. Which Group Has More High-demand Days?

```
1 high_working = (
2     data[(data["workingday"] == 1) & (data["cnt"] > threshold)].shape[0] /
3     data[data["workingday"] == 1].shape[0])
```

```
4 )
5
6 high_nonworking = (
7     data[(data["workingday"] == 0) & (data["cnt"] > threshold)].shape[0] /
8     data[data["workingday"] == 0].shape[0]
9 )
10
11 high_working, high_nonworking
```

Working: **51.2%** Non-working: **47.2%**

High-demand days are slightly more common on working days.

1.11 9. Maximum Number of Rentals and Its Characteristics

```
1 max_cnt = data["cnt"].max()
2 n_days_max = data[data["cnt"] == max_cnt].shape[0]
3
4 working_share = (
5     data[(data["cnt"] == max_cnt) & (data["workingday"] == 1)].shape[0]
6     / n_days_max
7 )
```

The maximum rental count is **8714**, occurring on a single day (which is a working day).

1.12 10. Average Rentals for High vs. Low Demand by Season

```
1 data["demand"] = np.where(data["cnt"] > threshold, "high", "low")
2
3 data.groupby(["season", "demand"])["cnt"].mean()
```

High-demand days average between **5550–6200** rentals, while low-demand days average only **2300–3900**.

This demonstrates strong seasonal variation but consistently large gaps between high and low demand.

2 Visual Data Analysis

In this section, we explore the distribution of all categorical variables in the dataset and compare how these distributions differ between high-demand and low-demand days. The categorical features analyzed are: `season`, `yr`, `mnth`, `holiday`, `weekday`, `workingday`, and `weathersit`.

These variables are treated as categorical because their numeric values represent discrete labels or qualitative categories rather than continuous measurements. For example, `season` encodes the four seasons of the year; `mnth` distinguishes months as calendar labels; `weekday` assigns labels 0–6 to days of the week; and `holiday` and `workingday` are binary indicators. The variable `weathersit` codes for qualitative weather conditions (clear, mist, rain, etc.). Although stored as integers, such codes have no numerical meaning and should be analyzed as categorical data.

2.1 Preparation of Categorical Data

```
1 categorical_cols = [  
2     "season", "yr", "mnth", "holiday",  
3     "weekday", "workingday", "weathersit"  
4 ]  
5  
6 threshold = data["cnt"].median()  
7 data["high_demand"] = (data["cnt"] > threshold).astype(int)
```

We first define the categorical variables and create a new binary indicator `high_demand` denoting whether the number of rentals exceeds the median. This will allow us to visualize category frequencies conditioned on demand level.

2.2 Category Frequencies Across the Whole Dataset

```
1 data_uniques = pd.melt(  
2     frame=data,  
3     value_vars=categorical_cols  
4 )  
5  
6 data_uniques = (  
7     pd.DataFrame(data_uniques.groupby(["variable", "value"])["value"].count())  
8     .sort_index(level=[0, 1])  
9     .rename(columns={"value": "count"})  
10    .reset_index()  
11 )  
12
```

```

13 sns.catplot(
14     x="variable",
15     y="count",
16     hue="value",
17     data=data_uniques,
18     kind="bar"
19 )
20 plt.xticks(rotation=90);

```

Figure 1 shows the frequency of each category for all categorical variables.

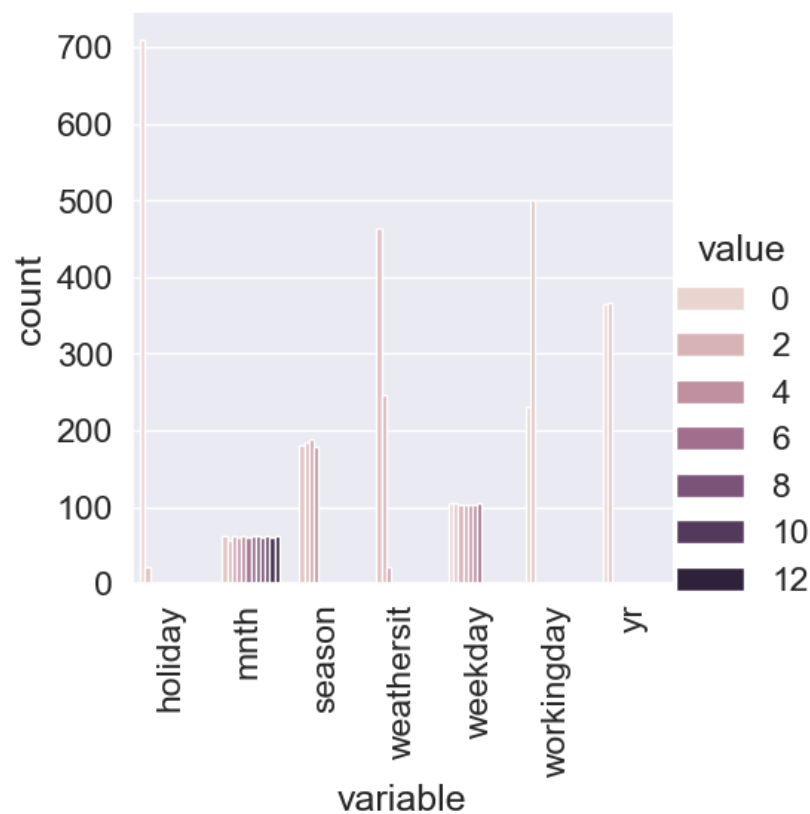


Figure 1: Counts of all categorical feature values in the dataset.

The distribution of categories is well balanced for weekday and season (see Figure 1). In contrast, variables such as holiday and weathersit are highly imbalanced, with very few holidays and very few days of severe weather.

2.3 Category Frequencies by Demand Level

```

1 data_uniques = pd.melt(
2     frame=data,
3     value_vars=categorical_cols,
4     id_vars=["high_demand"]
5 )
6
7 data_uniques = (
8     pd.DataFrame(data_uniques.groupby(["variable", "value", "high_demand"])["
9         value"].count())
10    .rename(columns={"value": "count"})
11    .reset_index()
12 )
13
14 sns.catplot(
15     x="variable",
16     y="count",
17     hue="value",
18     col="high_demand",
19     data=data_uniques,
20     kind="bar"
21 )
plt.xticks(rotation=90);

```

Figure 2 displays the same counts separately for low-demand (`high_demand = 0`) and high-demand (`high_demand = 1`) days.

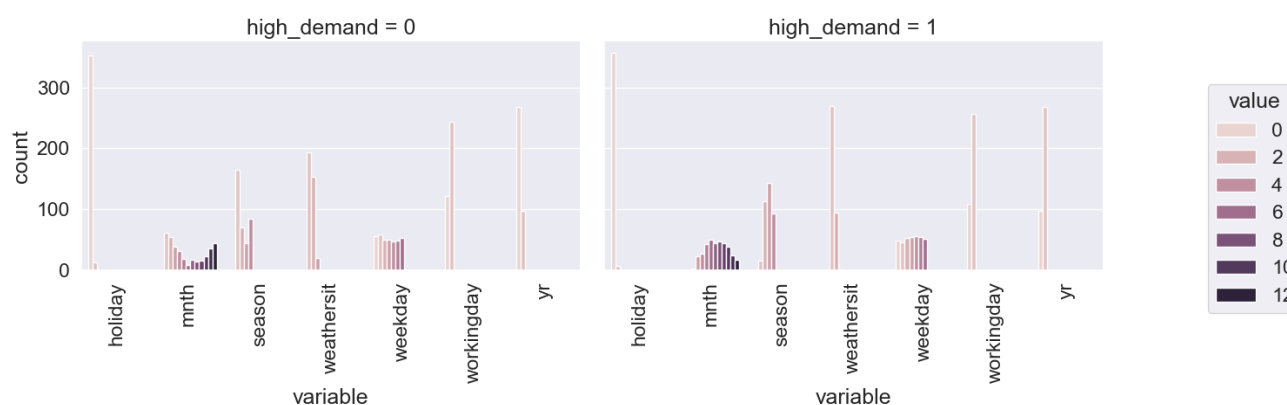


Figure 2: Categorical value counts separated by demand level.

The seasonal and monthly distributions differ noticeably in Figure 2: high-demand days appear

more frequently during summer months, while low-demand days are more common in winter. Other categories such as `holiday` and `workingday` show similar patterns in both groups, indicating that demand is not driven solely by calendar effects.

2.4 Uniqueness and Frequency of Each Feature

```
1 for c in data.columns:
2     n = data[c].nunique()
3     print(c)
4     if n <= 10:
5         print(n, sorted(data[c].value_counts().to_dict().items()))
6     else:
7         print(n)
8     print("_" * 10)
```

This inspection confirms that categorical features have low cardinality:

- `season` (4 categories),
- `yr` (2 categories),
- `holiday` (highly imbalanced: 710 non-holiday vs. only 21 holiday days),
- `weekday` (uniform distribution: 104–105 per day),
- `workingday` (500 working days and 231 non-working days),
- `weathersit` (463 clear, 247 mist, 21 light rain/snow days).

2.5 Summary of Feature Types

```
1 categorical_count = len(categorical_cols)
2 numerical_count = len(numerical_cols)
3
4 print("Categorical features ({}):".format(categorical_count))
5 for c in categorical_cols:
6     print("  ", c)
7
8 print("\nNumerical features ({}):".format(numerical_count))
9 for c in numerical_cols:
10    print("  ", c)
11
12 print("\nTotal rows:", data.shape[0])
```

The dataset contains 7 categorical and 9 numerical features, with a total of 731 daily observations.

2.6 Part 1: Basic Observations

2.6.1 Working vs. non-working days

First, I compared the number of working and non-working days using the binary feature `workingday`.

```
1 # 1.1 Number of working vs non-working days
2 data["workingday"].value_counts()
```

The output is:

```
workingday
1      500
0      231
Name: count, dtype: int64
```

Thus, the dataset contains 500 working days and 231 non-working (weekend/holiday) days.

Next, I computed the mean number of rentals (`cnt`) for working and non-working days:

```
1 # 1.2 Average rentals on working vs non-working days
2 data.groupby("workingday")["cnt"].mean()
```

which gives:

```
workingday
0    4330.168831
1    4584.820000
Name: cnt, dtype: float64
```

On average, working days have slightly higher bike demand than non-working days.

2.6.2 Share of high-demand days

I introduced a binary feature `high_demand` indicating whether daily demand exceeds the median:

```
1 threshold = data["cnt"].median()
2 data["high_demand"] = (data["cnt"] > threshold).astype(int)
```

Using this feature, I compared the share of high-demand days between working and non-working days:

```
1 # 1.3 Share of high-demand days for working vs non-working
2 share_work = data.loc[data["workingday"] == 1, "high_demand"].mean()
3 share_nonwork = data.loc[data["workingday"] == 0, "high_demand"].mean()
```

```
4
5 diff_pp = round(100 * (share_nonwork - share_work))
6 diff_pp
```

The result is -4, meaning that working days have about 4 percentage points *more* high-demand days than non-working days (the difference is small but noticeable).

2.6.3 Temperature difference between high and low demand

The feature `temp` is a normalized temperature in Celsius, with the transformation

$$T_C = \text{temp} \cdot (T_{\max} - T_{\min}) + T_{\min},$$

where $T_{\min} = -8^{\circ}\text{C}$ and $T_{\max} = 39^{\circ}\text{C}$. I computed the median temperature for high- vs. low-demand days in degrees Celsius:

```
1 # 1.4 Median temperature difference (°C) between high- and low-demand days
2 t_min, t_max = -8, 39
3
4 median_high = data.loc[data["high_demand"] == 1, "temp"].median()
5 median_low  = data.loc[data["high_demand"] == 0, "temp"].median()
6
7 median_high_c = median_high * (t_max - t_min) + t_min
8 median_low_c  = median_low  * (t_max - t_min) + t_min
9
10 round(median_high_c - median_low_c)
```

The resulting difference is positive, showing that high-demand days are warmer on average than low-demand days.

2.6.4 Risk mapping for summer working days

To study the effect of temperature in more detail, I focused on *summer working days*, i.e. rows with `season = 3` and `workingday = 1`:

```
1 # 1.2 Risk maps: summer working days
2 summer_work = data[(data["season"] == 3) & (data["workingday"] == 1)]
3 summer_work["temp"].describe()
```

Based on the first and third quartiles of `temp`, I defined “cool” and “warm” days and computed the fraction of high-demand days in each group:

```
1 q25 = summer_work["temp"].quantile(0.25)
2 q75 = summer_work["temp"].quantile(0.75)
```

```

3
4 print("Cool threshold:", q25)
5 print("Warm threshold:", q75)
6
7 p_cool = summer_work[summer_work["temp"] < q25]["high_demand"].mean()
8 p_warm = summer_work[summer_work["temp"] >= q75]["high_demand"].mean()
9
10 ratio = p_warm / p_cool
11 p_cool, p_warm, ratio

```

Example output:

```

Cool threshold: 0.665833
Warm threshold: 0.7516665
(0.71875, 0.6969697, 0.9696970)

```

Both probabilities are high and quite close, and the ratio is close to 1. This suggests that within already-warm summer working days, demand is consistently high and does not change much between “cool” and “very warm” conditions.

2.6.5 Utilization feature

I constructed a new feature measuring how intensively the system is used relative to the number of registered users:

$$\text{utilization} = \frac{\text{cnt}}{\text{registered} + 10^{-6}}.$$

```

1 # 1.3 New feature: utilization
2 data["utilization"] = data["cnt"] / (data["registered"] + 1e-6)
3
4 # Overall median
5 data["utilization"].median()
6
7 # Median utilization by workingday
8 data.groupby("workingday")["utilization"].median()
9
10 # Median utilization by high_demand
11 data.groupby("high_demand")["utilization"].median()
12
13 # Median utilization by workingday x high_demand
14 data.groupby(["workingday", "high_demand"])["utilization"].median()

```

The median utilization is higher on non-working days than on working days, and it is higher on high-demand days than on low-demand days. The strongest difference appears on non-working high-demand days, indicating that on busy weekends the system is used most intensively.

2.6.6 Outlier detection and cleaning

To reduce the influence of extreme or potentially erroneous values, I removed rows where humidity, wind speed, or total demand were outside their [2.5%, 97.5%] percentile range:

```
1 # 1.4 Cleaning data: remove extreme values in hum, windspeed, and cnt
2 to_remove = data[
3     (data["hum"] < data["hum"].quantile(0.025)) |
4     (data["hum"] > data["hum"].quantile(0.975)) |
5     (data["windspeed"] < data["windspeed"].quantile(0.025)) |
6     (data["windspeed"] > data["windspeed"].quantile(0.975)) |
7     (data["cnt"] < data["cnt"].quantile(0.025)) |
8     (data["cnt"] > data["cnt"].quantile(0.975))
9 ]
10
11 removed_share = to_remove.shape[0] / data.shape[0]
12 removed_share
13
14 filtered_data = data[~data.index.isin(to_remove.index)]
15 filtered_data.shape
```

About 15% of the observations are removed, leaving 621 clean days for subsequent visual analysis.

2.7 Part 2: Visual Data Analysis

For the visualizations below I use a working copy of the cleaned data:

```
1 # Use cleaned data for plots
2 df = filtered_data.copy()
```

2.7.1 Pearson correlation matrix

I examined linear relationships among the main numerical variables: temp, atemp, hum, windspeed, casual, registered, and cnt. The resulting Pearson correlation matrix is shown in Figure 3.

```
1 import numpy as np
2 import seaborn as sns
```

```

3 import matplotlib.pyplot as plt
4
5 num_cols = [
6     "temp", "atemp", "hum", "windspeed",
7     "casual", "registered", "cnt"
8 ]
9
10 # 2.1 Pearson correlation matrix
11 corr = df[num_cols].corr(method="pearson")
12
13 # mask upper triangle
14 mask = np.zeros_like(corr, dtype=bool)
15 mask[np.triu_indices_from(mask)] = True
16
17 f, ax = plt.subplots(figsize=(12, 9))
18 sns.heatmap(
19     corr,
20     mask=mask,
21     vmax=1,
22     center=0,
23     annot=True,
24     fmt=".2f",
25     square=True,
26     linewidths=0.5,
27     cbar_kws={"shrink": 0.5}
28 )
29 plt.tight_layout()
30 plt.savefig("corr_pearson_clean.png")
31 plt.show()

```

The strongest correlation is between registered and cnt; temperature and “feels-like” temperature also correlate positively with demand, while humidity and wind speed have weak negative correlations (see Figure 3).

2.7.2 Distribution of rentals on working vs. non-working days

To compare the distribution of daily rentals between working and non-working days, I used violin plots and kernel density estimates (Figures 4 and 5).

```

1 # 2.2 Violin plot for cnt by workingday
2 df_melt = pd.melt(frame=df, value_vars=["cnt"], id_vars=["workingday"])
3

```

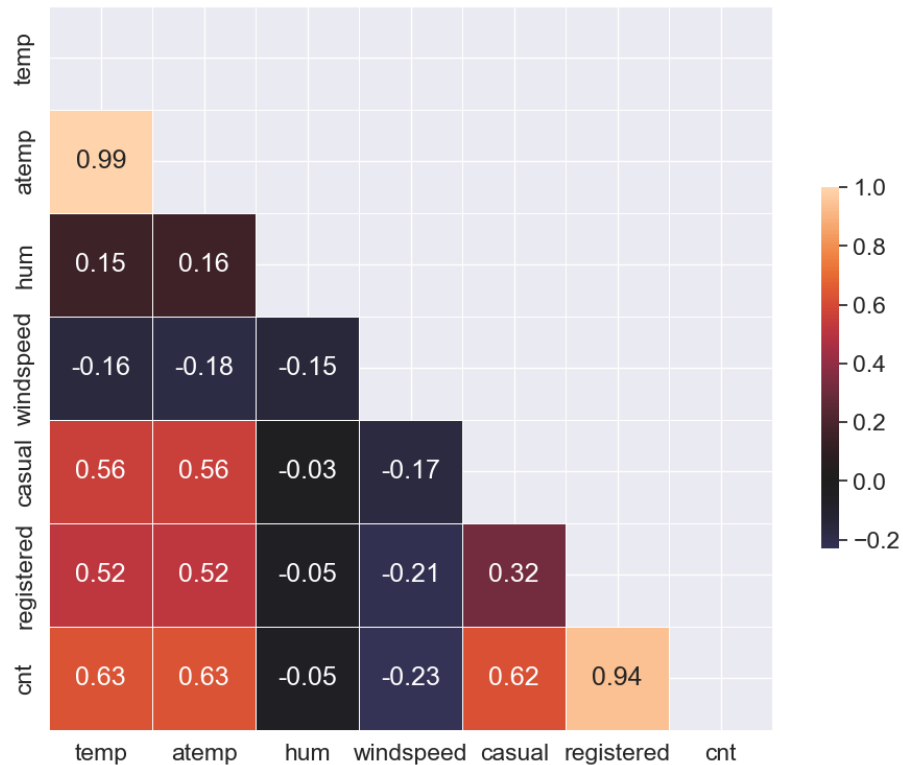


Figure 3: Pearson correlations between numerical variables (cleaned data).

```

4 plt.figure(figsize=(12, 10))
5 sns.violinplot(
6     x="variable",
7     y="value",
8     hue="workingday",
9     data=df_melt,
10    split=True,
11    scale="count",
12    scale_hue=False
13 )
14 plt.tight_layout()
15 plt.savefig("cnt_violin_working.png")
16 plt.show()
17
18 # 2.2 KDE plot for cnt by workingday
19 g = sns.FacetGrid(df, hue="workingday", height=3)
20 g.map(sns.kdeplot, "cnt").add_legend()

```

```
21 plt.tight_layout()
22 plt.savefig("cnt_kde_working.png")
23 plt.show()
```

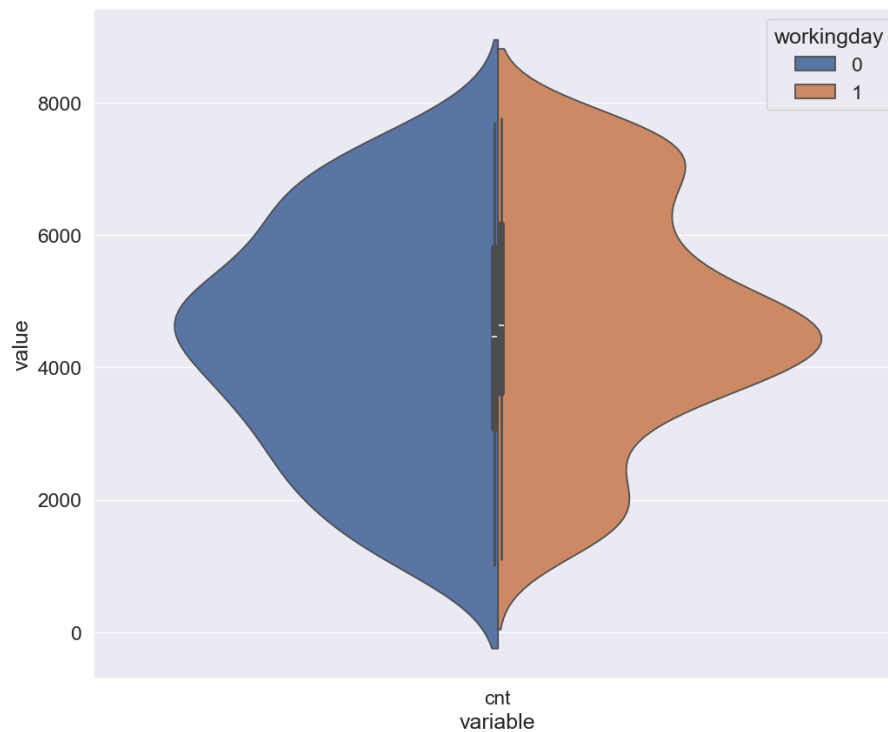


Figure 4: Distribution of daily rentals for working vs. non-working days.

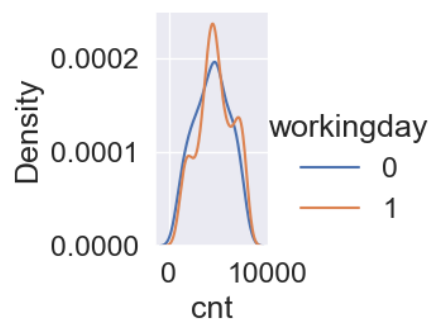


Figure 5: Kernel density of daily rentals for working vs. non-working days.

Both plots show similar shapes for the two groups, but the working-day distribution is slightly shifted to higher values, consistent with the higher mean demand observed earlier (Figures 4 and 5).

2.7.3 Spearman rank correlation

I also computed the Spearman rank correlation matrix to assess monotonic relationships that may not be strictly linear; the result is shown in Figure ??.

2.7.4 Monthly pattern of high demand

Finally, I visualized how high-demand days are distributed over the months (Figure 6):

```
1 # 2.4 Month vs high demand
2 sns.countplot(x="mnth", hue="high_demand", data=df)
3 plt.tight_layout()
4 plt.savefig("mnth_high_demand_counts.png")
5 plt.show()
```

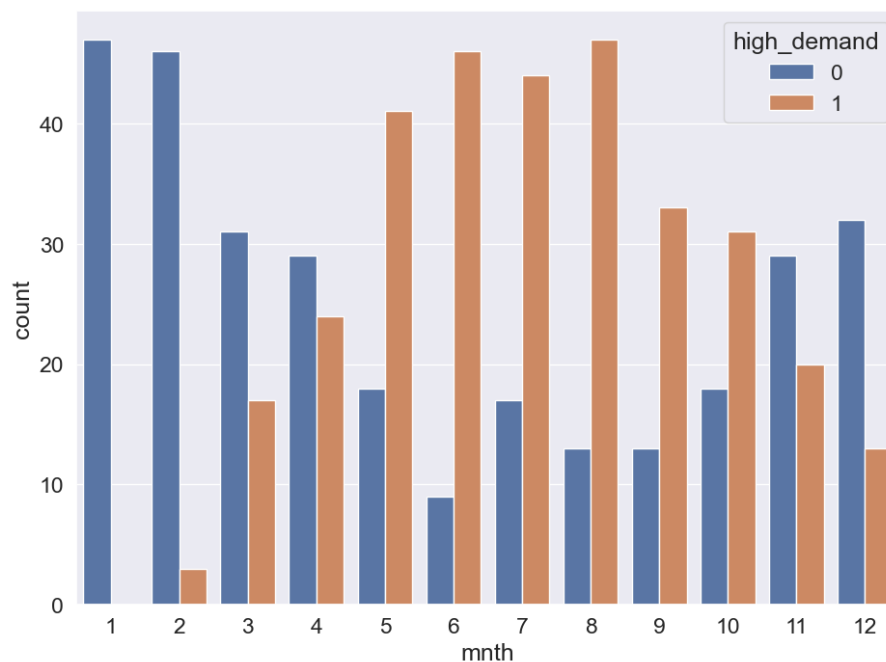


Figure 6: Counts of high- and low-demand days by month.

High-demand days cluster in late spring, summer, and early autumn, whereas winter months have more low-demand days, highlighting the strong seasonality of bike usage (Figure 6).

3 Classification, Decision Trees and k Nearest Neighbors

In this section, multiple machine learning models are applied to the processed dataset to predict whether a day is a *high-demand* or a *low-demand* day, based on calendar, weather, and usage information. In addition, a separate regression tree is trained to directly predict the numerical rental count (cnt). This section includes decision trees, random forests, k -nearest neighbours, and a regression tree model.

The target variable for classification is defined as:

$$\text{high_demand} = \begin{cases} 1, & \text{if cnt} > \text{median}(\text{cnt}), \\ 0, & \text{otherwise.} \end{cases}$$

The final distribution is balanced (366 low-demand days, 365 high-demand days).

3.1 Train–Test Split

The dataset is divided using stratified sampling to preserve the proportion of high- and low-demand days:

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.3, random_state=17, stratify=y  
3 )
```

This produces 511 training and 220 testing samples.

3.2 Decision Tree Classifier

A baseline decision tree classifier with `max_depth=3` achieves:

$$\text{Accuracy} = 0.836.$$

Decision trees recursively partition the space by choosing splits that maximise impurity reduction. For classification, the Gini impurity is used:

$$G = 1 - \sum_{k=1}^K p_k^2,$$

where p_k is the fraction of samples belonging to class k .

The shallow decision tree reveals interpretable decision rules. Figure 7 visualises the learned tree. The most important splits involve:

- `instant` (proxy for time in year),

- temp,
- hum.

Low values of instant and low temperatures are associated with low demand, whereas late-season days and moderate humidity favour high-demand classification.

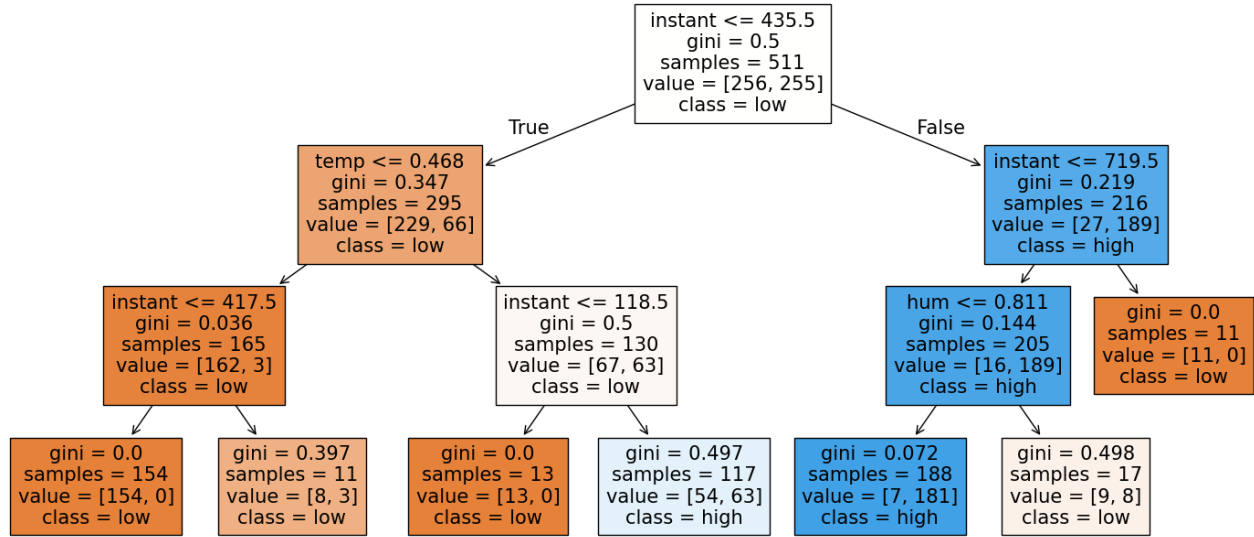


Figure 7: Decision Tree Classifier (depth=3) predicting high- vs. low-demand days.

3.3 Hyperparameter-Tuned Decision Tree

A grid search over maximum depth 2–10 identifies:

best depth = 6, best cross-validation score = 0.8747.

The tuned tree reaches:

Test accuracy = 0.882.

The deeper model captures more nonlinear behaviour but remains interpretable.

3.4 Random Forest Classifier

Random forests combine many decision trees using bootstrap aggregation. Each tree receives a random subset of features, making the ensemble less correlated. The prediction is the majority vote:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_M(x)).$$

The baseline forest (100 trees) achieves:

$$\text{Accuracy} = 0.914.$$

Cross-validation confirms strong generalisation performance:

$$\text{CV mean accuracy} = 0.873.$$

A tuned forest using GridSearchCV selects:

$$\text{max_depth} = 10, \quad \text{max_features} = 10.$$

The tuned model attains:

$$\text{Accuracy} = 0.905.$$

3.5 k -Nearest Neighbours Classifier

The k -NN method classifies a point based on the class distribution of its closest neighbours:

$$\hat{y}(x) = \text{mode}(y_{(1)}, y_{(2)}, \dots, y_{(k)}),$$

where distances are computed in the scaled feature space.

Using $k = 10$, the model achieves:

$$\text{Accuracy} = 0.868.$$

Grid search identifies the optimal k :

$$k=9, \quad \text{CV accuracy}=0.863,$$

with a test accuracy of:

$$\text{Accuracy} = 0.859.$$

3.6 Decision Tree Regressor for Predicting cnt

A decision tree regressor ($\text{max_depth} = 5$) is trained to predict the numerical rental count. The regression trees minimise the mean squared error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Figure 8 shows how the Decision Tree Regressor (with a maximum depth of 5) performs when predicting the daily number of bike rentals. The plot compares the predicted values with the actual

observed counts, and the diagonal $y = x$ line represents perfect prediction. Every point lying exactly on this line would correspond to a day for which the model predicted the demand perfectly.

In practice, the points are scattered around the diagonal, and a clear pattern emerges. The model manages to capture the overall trend: days with high demand tend to be predicted as such, and low-demand days are also generally recognised. However, the predictions show a noticeable bias toward the middle range. Very busy days tend to be underestimated (the points fall below the line on the right side of the plot), while days with unusually low demand are often overestimated (their points appear above the line on the left). In other words, the model smooths the extremes and avoids making very high or very low predictions.

This behaviour is typical for relatively shallow decision trees. With limited depth, the regressor cannot form very detailed partitions of the feature space, so it replaces complex patterns with a few broad average values. As a result, the model is reasonably good at capturing the general structure of the data but struggles with the day-to-day variability that characterises real-world bike usage.

The mean squared error (MSE) of the model is

$$\text{MSE} = 835,756.02,$$

which reflects this behaviour: the model is not completely off, but it does make sizeable errors, especially on days with unusually high or low demand.

3.7 Model Performance Comparison

Model	Validation Score	Test Accuracy / MSE
Decision Tree (depth=3)	–	0.836
Tuned Decision Tree (depth=6)	0.875	0.882
Random Forest (100 trees)	0.873	0.914
Tuned Random Forest	0.869	0.905
k -NN ($k = 10$)	–	0.868
Tuned k -NN ($k = 9$)	0.863	0.859
Decision Tree Regressor	–	MSE = 835,756

Table 1: Comparison of model performance across classifiers and regressors

3.8 Summary

Random Forests achieve the highest test accuracy (0.914), outperforming both decision trees and k -NN. The tuned decision tree improves significantly over the baseline, showing that model depth plays an important role. The regression tree demonstrates moderate predictive performance but reveals a clear linear trend between predicted and true demand (Figure 8).

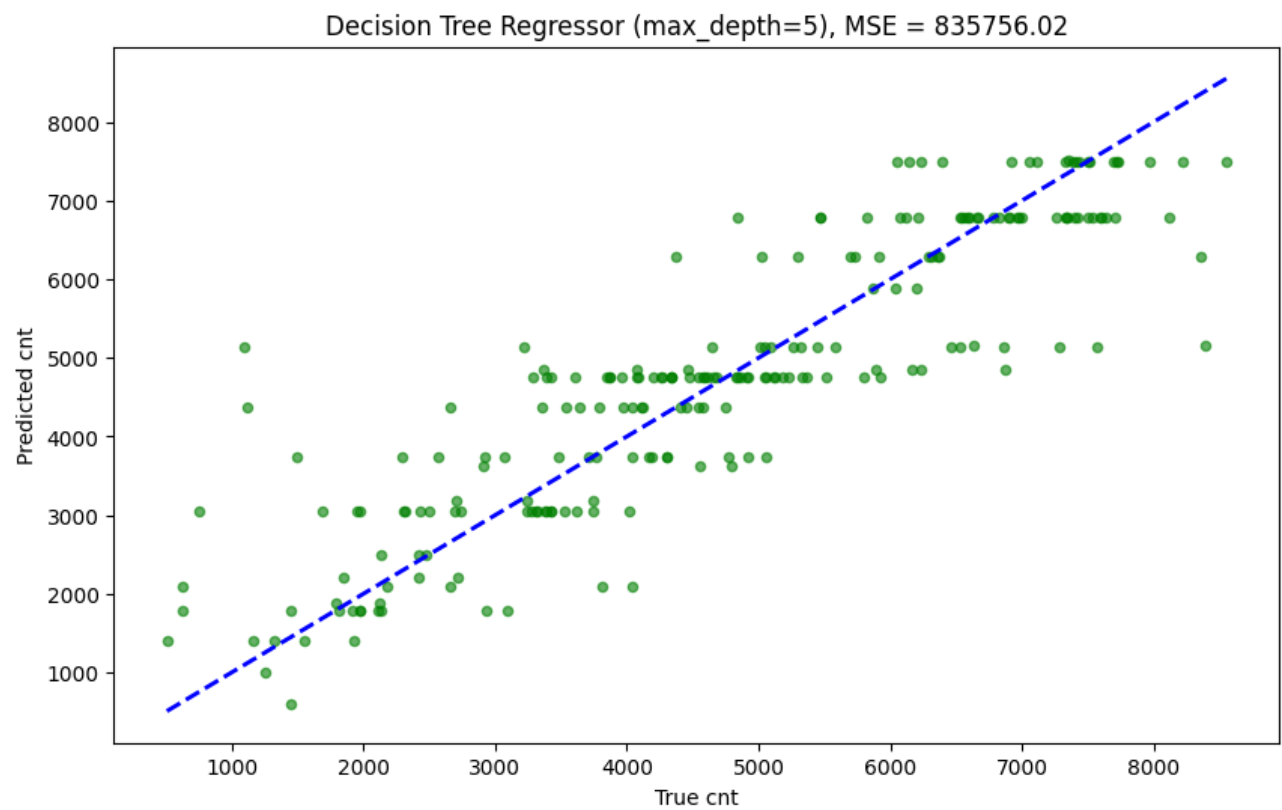


Figure 8: Decision Tree Regressor: predicted vs. true demand values.

4 Linear Classification

The target variable was the total number of bike rentals

$$y = \text{cnt},$$

and the feature matrix X was constructed from all remaining variables except `instant`, `dteday`, `registered`, and `casual`:

$$X = \text{data} \setminus \{\text{cnt}, \text{instant}, \text{dteday}, \text{registered}, \text{casual}\}.$$

The data was split into training and holdout sets:

$$\text{train_size} = 0.7, \quad \text{test_size} = 0.3, \quad \text{random_state} = 17.$$

All features were standardized using `StandardScaler`:

$$\tilde{X}_{\text{train}} = \text{StandardScaler.fit_transform}(X_{\text{train}}), \quad \tilde{X}_{\text{test}} = \text{StandardScaler.transform}(X_{\text{test}}).$$

Model quality was evaluated using the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

4.1 Ordinary Least Squares Linear Regression

The OLS model estimates the coefficient vector β by minimizing the sum of squared errors:

$$\hat{\beta}_{\text{OLS}} = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2.$$

After training on the standardized features, we obtain:

$$\text{MSE}_{\text{train}} \approx 660,548, \quad \text{MSE}_{\text{test}} \approx 993,749.$$

The largest coefficients (in absolute value) correspond to `yr`, `season`, `atemp`, and `temp`. Negative contributions come from `weathersit`, `hum`, and `windspeed`. The features `holiday` and `workingday` have the smallest influence.

4.2 Lasso Regression

Lasso introduces L_1 regularization, which encourages sparse coefficient vectors:

$$\hat{\beta}_{\text{Lasso}} = \arg \min_{\beta} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - x_i^\top \beta)^2 + \alpha \|\beta\|_1 \right], \quad \|\beta\|_1 = \sum_{j=1}^p |\beta_j|.$$

First we trained a Lasso model with fixed $\alpha = 0.01$. Then we tuned α using LassoCV over the grid $\alpha \in (-6, 2)$. The optimal value was:

$$\alpha^* \approx 2.705.$$

For the tuned Lasso model:

$$\text{MSE}_{\text{train}} \approx 660,676, \quad \text{MSE}_{\text{test}} \approx 989,136.$$

Feature importance (via coefficient magnitude) is consistent with OLS: yr, season, atemp, and temp dominate.

4.3 Random Forest Regression

A Random Forest is an ensemble of decision trees. The prediction is the average of the tree outputs:

$$\hat{y}(x) = \frac{1}{T} \sum_{t=1}^T f_t(x).$$

We trained a baseline RandomForestRegressor (random_state = 17). The results are:

$$\text{MSE}_{\text{train}} \approx 68,005, \quad \text{MSE}_{\text{cv}} \approx 524,138, \quad \text{MSE}_{\text{test}} \approx 440,696.$$

Next, we performed hyperparameter tuning using GridSearchCV with:

$$\text{max_depth} \in \{10, \dots, 24\}, \quad \text{max_features} \in \{6, \dots, 11\}.$$

The best model achieved:

$$\text{MSE}_{\text{cv}} \approx 494,264, \quad \text{MSE}_{\text{test}} \approx 440,392.$$

Thus, Random Forest substantially outperforms linear models.

4.4 Model Comparison

Table 2 summarizes the performance of all models.

It is clear that the linear models (OLS and Lasso) perform similarly and significantly worse than the Random Forest on the test set. The tuned Random Forest offers the best cross-validation and test performance.

Table 2: Comparison of models by MSE

Model	MSE (train)	MSE (cv)	MSE (test)
Linear Regression (OLS)	660,548	–	993,749
Lasso (LassoCV, $\alpha^* \approx 2.705$)	660,676	–	989,136
Random Forest (baseline)	68,005	524,138	440,696
Random Forest (tuned)	–	494,264	440,392

4.5 Feature Importance Analysis

For linear models, feature importance is defined as the absolute value of coefficients:

$$\text{importance}_j^{\text{lin}} = |\hat{\beta}_j|.$$

The most influential predictors are: yr, season, atemp, temp. The least influential are holiday and workingday.

Random Forest uses impurity-based importance:

$$\text{importance}_j^{\text{RF}} = \frac{1}{T} \sum_{t=1}^T \Delta \text{MSE}_{t,j},$$

where $\Delta \text{MSE}_{t,j}$ is the reduction in node impurity from splits on feature j .

The most important features are: temp, yr, atemp, followed by season and hum. The least important are again workingday and holiday, consistent with linear models.

5 Bagging and Random Forest

To study demand classification, we converted the original count variable `cnt` into a binary target. We thresholded the total number of rentals at its median value:

$$\text{threshold} = \text{median}(\text{cnt}), \quad y = I\{\text{cnt} > \text{threshold}\},$$

where $I\{\cdot\}$ is the indicator function. The resulting target is stored in the column `high_demand`.

The distribution of the target is almost perfectly balanced:

$$P(y = 0) \approx 0.501, \quad P(y = 1) \approx 0.499.$$

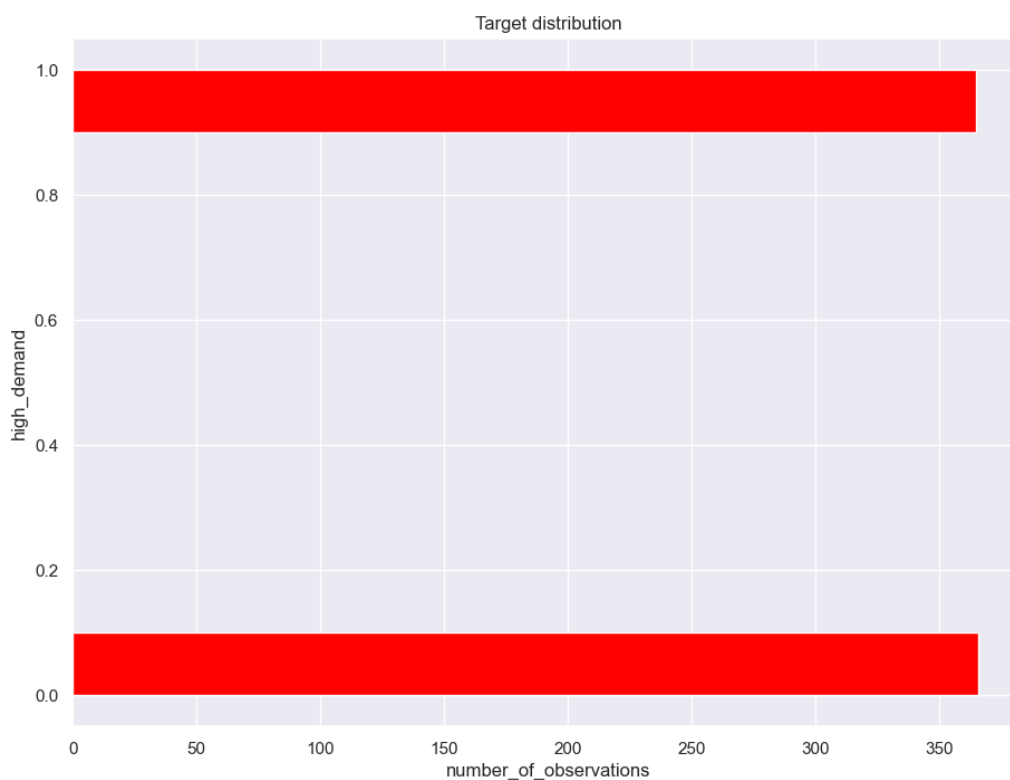


Figure 9: Distribution of the binary target `high_demand`.

Categorical features used in the models were `{season, yr, mnth, holiday, weekday, workingday, weathersi}` and numerical features were `{instant, temp, atemp, hum, windspeed}`. The full feature matrix is therefore

$$X = \text{data}[\text{independent_columns_names}], \quad y = \text{high_demand}.$$

5.1 Bootstrapping the Mean Temperature on High-Demand Days

To assess the uncertainty of the mean temperature on high-demand days, we applied non-parametric bootstrapping. Let $\{z_1, \dots, z_n\}$ be the observed temperatures on days with `high_demand` = 1. A bootstrap sample is obtained by sampling with replacement from this set:

$$(z_1^*, \dots, z_n^*) \sim \{z_1, \dots, z_n\} \text{ with replacement.}$$

We generated $B = 1000$ bootstrap samples. For each sample we computed the sample mean

$$\bar{z}_b^* = \frac{1}{n} \sum_{i=1}^n z_{i,b}^*, \quad b = 1, \dots, B.$$

A $(1 - \alpha)$ -level bootstrap percentile confidence interval for the mean is then obtained as

$$[\hat{\theta}_{\alpha/2}, \hat{\theta}_{1-\alpha/2}] = [\text{Quantile}_{\alpha/2}(\{\bar{z}_b^*\}), \text{Quantile}_{1-\alpha/2}(\{\bar{z}_b^*\})].$$

For $\alpha = 0.1$ (i.e. a 90% confidence interval), we obtained:

$$\text{CI}_{0.9}(E[\text{temp} \mid y = 1]) \approx [0.581, 0.603].$$

5.2 Logistic Regression

Logistic regression models the conditional probability of the positive class as

$$P(y = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + \exp(-(w^\top x + b))},$$

where w is the weight vector, b is the bias term, and $\sigma(\cdot)$ is the logistic sigmoid.

The parameters are estimated by minimizing the regularized log-loss:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \lambda \|w\|_2^2,$$

where $p_i = P(y_i = 1 \mid x_i)$ and λ is a regularization parameter. In the implementation, the inverse regularization strength is denoted by $C = 1/\lambda$.

We tuned the parameter C using grid search with stratified 5-fold cross-validation, optimizing ROC AUC. The best configuration was

$$C^* = 10, \quad \text{ROC AUC}_{\text{cv}} \approx 0.936,$$

with a cross-validation standard deviation $\text{std} \approx 0.024$.

After standardizing features and refitting the model with C^* , we analyzed feature importance via the absolute values of the coefficients. Table 3 lists the coefficients.

We see that the most influential predictors for separating high- and low-demand days are `yr`, `temp`, and `instant`, followed by `season` and `atemp`. Calendar indicators `holiday` and `workingday` have comparatively small impact.

Table 3: Logistic regression coefficients sorted by absolute value.

Feature	Coefficient	Coefficient
yr	3.242001	3.242001
temp	2.765620	2.765620
instant	-1.456610	1.456610
season	0.993482	0.993482
atemp	-0.702903	0.702903
mnth	0.666353	0.666353
hum	-0.519769	0.519769
weathersit	-0.512044	0.512044
windspeed	-0.474140	0.474140
weekday	0.173920	0.173920
holiday	-0.141699	0.141699
workingday	0.031012	0.031012

5.3 Ensembles, Bootstrapping, and Bagging

Ensemble methods combine multiple base models to obtain a final predictor with lower variance and/or bias than individual models. A generic ensemble classifier aggregates the predictions of T base learners $\{h_1, \dots, h_T\}$:

$$\hat{y}(x) = \text{majority_vote}(h_1(x), \dots, h_T(x)),$$

or, in probabilistic form,

$$\hat{p}(y = 1 \mid x) = \frac{1}{T} \sum_{t=1}^T h_t(x).$$

Bootstrapping. Bootstrapping is the core resampling technique used in many ensembles. Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, a bootstrap sample $\mathcal{D}^*$ is obtained by sampling n points from $\mathcal{D}$ *with replacement*. This procedure is repeated independently to obtain multiple bootstrap copies of the original data.

Bagging. Bootstrap aggregating, or bagging, uses bootstrapping to train multiple versions of a base model and then averages their predictions. The standard bagging algorithm can be summarized as:

1. Draw T bootstrap samples $\mathcal{D}_1^*, \dots, \mathcal{D}_T^*$ from the original dataset.
2. Train a base learner h_t on each $\mathcal{D}_t^*$.

-
3. For a new input x , compute the aggregated prediction

$$\hat{f}_{\text{bag}}(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

(for classification, this is typically converted into a class label via majority vote).

Bagging reduces variance of unstable base learners by averaging over many bootstrap replicates.

5.4 Random Forest Classifier

Random Forest extends bagging of decision trees with an additional layer of randomization: at each node split, a random subset of features is considered. For each tree t , at each split a subset of $m = \text{max_features}$ predictors is drawn from the full set of p predictors, and the best split is selected only among those m features.

In our experiments, we used a `RandomForestClassifier` with class weights balanced, and tuned its hyperparameters via grid search with stratified 5-fold cross-validation. The tuning grid included:

$$\text{max_features} \in \{1, 2, 4\}, \quad \text{min_samples_leaf} \in \{3, 5, 7, 9\}, \quad \text{max_depth} \in \{5, 10, 15\}.$$

The best model achieved

$$\text{ROC AUC}_{\text{cv}} \approx 0.967,$$

which substantially improves upon logistic regression. The difference in mean ROC AUC is approximately

$$\Delta\text{AUC} = \text{AUC}_{\text{RF}} - \text{AUC}_{\text{LogReg}} \approx 0.9668 - 0.9361 \approx 0.031.$$

Feature importance in Random Forest is computed via the average decrease in impurity (e.g. Gini or entropy) across all trees and splits. For a feature j , its importance is defined as

$$\text{importance}_j = \frac{1}{T} \sum_{t=1}^T \sum_{s \in \mathcal{S}_{t,j}} \Delta\text{Impurity}(s),$$

where $\mathcal{S}_{t,j}$ is the set of splits on feature j in tree t .

The resulting importances are shown in Table 4.

We again observe that `instant`, `atemp`, `temp`, and `yr` are the most informative predictors of high demand, while `holiday` and `workingday` have negligible importance.

Table 4: Random Forest feature importances sorted in descending order.

Feature	Importance
instant	0.301861
atemp	0.175866
temp	0.165032
yr	0.100719
hum	0.063678
season	0.055651
windspeed	0.044943
mnth	0.036689
weathersit	0.029132
weekday	0.021256
workingday	0.004611
holiday	0.000561

5.5 Bagging with Logistic Regression

Finally, we applied bagging to logistic regression. The base estimator is a logistic regression model with class weights balanced:

$$h_t(x) = \sigma(w_t^\top x + b_t),$$

trained on each bootstrap sample. The aggregated prediction of the bagging ensemble is

$$\hat{p}_{\text{bag}}(y = 1 \mid x) = \frac{1}{T} \sum_{t=1}^T h_t(x),$$

with classification based on a threshold (typically 0.5).

We used a `BaggingClassifier` with $T = 100$ base learners and performed `RandomizedSearchCV` over the parameters

`max_features` $\in \{2, 3, 4\}$, `max_samples` $\in \{0.5, 0.7, 0.9\}$, `estimator__C` $\in \{10^{-4}, 10^{-3}, 10^{-2}, 1, 10, 100\}$.

The best bagging configuration achieved

$$\text{ROC AUC}_{\text{cv}} \approx 0.936,$$

which is comparable to, but slightly below, the performance of the single tuned logistic regression model and clearly worse than the Random Forest.

Table 5: ROC AUC comparison of classifiers (5-fold cross-validation).

Model	Mean ROC AUC	Comments
Logistic Regression (tuned)	0.936	$C^* = 10$, std ≈ 0.024
Bagging(Logistic Regression)	0.936	Similar to single LR, no clear gain
Random Forest (tuned)	0.967	Best performance, $\Delta\text{AUC} \approx 0.031$ vs LR

5.6 Comparison of Ensemble Methods

Table 5 summarizes the ROC AUC performance of the different models.

Overall, the Random Forest ensemble, which combines both bootstrapping and random feature selection, provides the highest discriminative power for predicting high-demand days. Bagging improves model stability but, when used with logistic regression as a base learner, does not significantly outperform a single, well-regularized logistic regression model.

6 Feature Engineering and Feature Selection

Feature engineering is the process of transforming raw data into meaningful representations suitable for machine learning. It includes three closely related tasks: *feature extraction*, *feature transformation*, and *feature selection*. Below is a concise overview of the main ideas.

6.1 Feature Extraction

Feature extraction refers to deriving numerical features from raw, unstructured, or heterogeneous data sources.

- **Texts:** Tokenization, lemmatization/stemming, Bag-of-Words, TF-IDF, n-grams, and modern embedding methods (Word2Vec, GloVe, FastText).
- **Images:** Using pretrained convolutional neural networks to obtain vector embeddings; extracting handcrafted descriptors such as edges, color statistics, brightness, or EXIF metadata.
- **Geospatial data:** Features based on coordinates (latitude/longitude), distances, proximity to points of interest (POIs), neighborhood statistics, or results of geocoding/reverse geocoding.
- **Date and time:** Deriving day of week, hour, month, holiday indicators, seasonality, cyclic encodings (sine/cosine), and time-based flags such as “is_weekend”.
- **Time series and web data:** Lag features, rolling means/variances, trends, user-agent parsing, device/browser type, IP-based geo-features, and session-level statistics.

6.2 Feature Transformations

Feature transformation modifies existing features to make them more compatible with machine learning models or to reduce noise and skewness.

- **Normalization and distribution adjustment:** Standard scaling, min–max scaling, logarithmic transformation, Box–Cox and Yeo–Johnson transforms to reduce heavy tails or skewness.
- **Interactions:** Constructing new variables such as ratios (e.g. price per room), products, polynomial features, or domain-specific combinations of variables.
- **Handling missing values:** Mean/median imputation, mode imputation for categoricals, adding “missing” indicators, forward/backward filling for ordered data, or using extreme values to let tree-based models split on missingness.

6.3 Feature Selection

Feature selection aims to reduce dimensionality by removing uninformative, noisy, or redundant variables, improving model interpretability and generalization.

- **Statistical approaches:** Dropping constant or low-variance features, applying univariate tests (ANOVA F-test, chi-square) to retain only features with statistically significant relationships to the target.
- **Selection by modeling:** Using models that estimate feature importance such as Lasso logistic regression (which drives some coefficients to zero) or Random Forest (which ranks features by impurity reduction).
- **Grid or search-based approaches:** Exhaustive Feature Selection (testing many feature subsets) or Sequential Feature Selection (forward or backward), evaluating each subset using cross-validation and selecting the best-performing one.

Feature engineering is a crucial step in building effective machine learning pipelines, often yielding greater performance improvements than complex model architectures.

7 Applied Feature Engineering and Feature Selection

In this section, we summarize the feature engineering and selection procedures applied to the Bike Sharing dataset and show the corresponding numerical results.

7.1 Date and Time Feature Extraction

We converted the `dteday` column to a datetime format and extracted calendar-based features:

$$\text{dow} = \text{weekday}(\text{dteday}), \quad \text{is_weekend} = I\{\text{dow} \in \{5, 6\}\}.$$

Examples of the resulting features for the first rows:

dteday	weekday	dow	is_weekend
2011-01-01	6	5	1
2011-01-02	0	6	1
2011-01-03	1	0	0

7.2 Cyclical Encoding of Periodic Features

To encode cyclic variables (month and weekday), we used harmonic sine–cosine transformations:

$$\begin{aligned}\text{mnth_cos} &= \cos\left(\frac{2\pi \text{mnth}}{12}\right), & \text{mnth_sin} &= \sin\left(\frac{2\pi \text{mnth}}{12}\right), \\ \text{weekday_cos} &= \cos\left(\frac{2\pi \text{dow}}{7}\right), & \text{weekday_sin} &= \sin\left(\frac{2\pi \text{dow}}{7}\right).\end{aligned}$$

For the first five observations we obtained, for example:

mnth	mnth_cos	mnth_sin	dow	weekday_cos	weekday_sin
1	0.962654	0.270734	5	−0.222521	−0.974928
1	0.962654	0.270734	6	0.623490	−0.781831
1	0.962654	0.270734	0	1.000000	0.000000
1	0.962654	0.270734	1	0.623490	0.781831
1	0.962654	0.270734	2	−0.222521	0.974928

We also checked that the harmonic encoding preserves cyclic distances. For example, Euclidean distances in the encoded space satisfy

$$d(23, 1) \approx 0.518, \quad d(9, 11) \approx 0.518, \quad d(9, 21) = 2.0,$$

showing that hours close on the clock remain close in the encoded space.

7.3 Distribution Transformations of the Target

Let cnt denote the daily number of rentals. We considered:

- the original values $x = \text{cnt}$,
- the standardized values

$$z = \frac{x - \mu}{\sigma},$$

- the min–max scaled values

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

- the log-transformed values

$$x_{\log} = \log(x + 1).$$

Using the Shapiro-Wilk test on a subset of 500 observations, we obtained:

$$\text{Shapiro}(\text{cnt}) : W = 0.9842, p \approx 2.87 \cdot 10^{-5},$$

$$\text{Shapiro}(\log(\text{cnt} + 1)) : W = 0.9112, p \approx 1.61 \cdot 10^{-16}.$$

The log transform visibly reduces right skewness, although strict normality is not achieved.

The first five values after each transform are:

StandardScaler	MinMaxScaler	$\log(\text{cnt} + 1)$
-1.818	0.111	6.894
-1.913	0.090	6.687
-1.630	0.153	7.208
-1.520	0.177	7.354
-1.500	0.182	7.378

7.4 Variance-Based Feature Selection (Synthetic Example)

On synthetic data from `make_classification`, we evaluated `VarianceThreshold` with different thresholds. For 100 samples and 20 initial features:

Threshold	Resulting shape
0.7	(100, 20)
0.8	(100, 20)
0.9	(100, 13)

Thus, a higher threshold removed 7 low-variance features.

7.5 Univariate Feature Selection (SelectKBest)

We applied `SelectKBest` with the ANOVA F-test, selecting $k = 5$ features, and compared logistic regression performance (5-fold CV, negative log-loss):

Model	Mean neg_log_loss
LR (all features)	-0.2741
LR (<code>SelectKBest</code> , $k = 5$)	-0.2312
LR (<code>VarianceThreshold</code> , $\text{thr}=0.9$)	-0.2502

Feature selection by ANOVA improved the loss compared to using all features.

7.6 Model-Based Feature Selection with Random Forest

We trained a Random Forest classifier and used `SelectFromModel` to choose informative features, then fitted logistic regression on this subset. For comparison, we report 5-fold CV negative log-loss:

Model	Mean neg_log_loss
LR (all features)	-0.2741
RF (all features)	-0.9789
LR on RF-selected features	-0.2404

Including standardization in the pipeline yields:

Pipeline	Mean neg_log_loss
StandardScaler + LR	-0.2682
StandardScaler + SelectFromModel(RF) + LR	-0.2400
RF (all features)	-0.9789

Model-based selection improves logistic regression performance, and Random Forest itself achieves the lowest loss on the synthetic data.

7.7 Summary

In summary, on the real Bike Sharing dataset we extracted calendar-based features such as `dow` and `is_weekend`, encoded cyclic variables (month and weekday) using harmonic sine cosine transformations, and examined the behaviour of the target distribution under several normalization techniques including standard scaling, minâmax scaling, and logarithmic transformation. Using synthetic data, we further demonstrated how feature selection techniques operate in practice: low-variance filtering removed uninformative predictors, univariate ANOVA selection identified the most statistically relevant features, and model-based selection with Random Forest highlighted informative variables and improved the downstream performance of logistic regression. Overall, these experiments illustrate how thoughtful feature engineering and feature selection reshape the statistical properties of data and have a direct and significant impact on model quality and robustness.

References

- Lakshmi N. Pathi. *Bike Sharing Dataset*. Kaggle. Available at: <https://www.kaggle.com/datasets/lakshmi25npathi/bike-sharing-dataset/data>
- Yury Kashnitsky et al. *mlcourse.ai â Open Machine Learning Course*. Available at: <https://mlcourse.ai/book/index.html>